SteeringLib: A Movement Behavior Library using SFML

Prem Gandhi[1]

Abstract— **Movement behavior algorithms are a crucial part of enabling creative and fun game mechanics for gamers. This report presents my approach for constructing SteeringLib, a steering behavior library using the Simple Fast Media Library (SFML). The approach utilizes virtual classes and simple calculations to avoid common inheritance issues with the diamond problem and computational complexity, respectively. The steering behaviors implemented in this paper are all based on kinematic behavior i.e. velocity/position and rotation/orientation are set directly.**

I. INTRODUCTION

Movement behavior algorithms serve as the fundamental bedrock for most games played by gamers, and some games rely almost exclusively on movement algorithms. Movement algorithms all take the same form by taking geometric data about the state of a character and then applying it in the context of a world frame. This paper will detail the design choices involved in the high-level software architecture, algorithm design, and parameter choices. These algorithms are colloquially called steering behaviors, the name given to Craig Reynold's movement algorithms. This paper explains the choices involved with choosing certain parameter values for steering behaviors and for the Boids algorithm.

II. SOFTWARE ARCHITECTURE

The project source files were split up by main source files, kinematic steering behaviors, and delegated behaviors. The kinematic steering behaviors were all subclassed from the KinematicMovement class. The delegated behaviors were extensions of the kinematic steering behaviors. This allows the primitive behaviors to be easily distinguished from the composite behaviors.

The SteeringLib implements a basic set of objects to hold and represent crucial geometric and game state data about characters.

1) Static: This object represents the position (a 2D vector) and orientation (an angle object) data for a game object. It provides basic helper functions for getting and setting private variables. Additionally, this object contains functions for equality comparison and multiple constructors for convenience.
2) KinematicSteeringOutput: This object represents the steering-related data for a game object, notably: velocity (a 2D vector) and rotation (an angle object). This object like the Static object above contains getter, setter, multiple constructor, and even a equality comparison functions.
3) KinematicMovement: This object represents a pure virtual class that will be subclassed by various steering behavior classes such as Align, Arrive, Flee, etc. The class contains basic helper functions and member variables such as getNewOrientation(), target, speed, etc.

The boid game object is designed using a class structure which holds a smart pointer to a newly created KinematicMovement subclass. This architecture enables the program to easily manage memory by discarding old pointers to objects when a new steering behavior is loaded into the boid. The boid game object holds other objects which enable it to place breadcrumbs and take user input by polling whether the mouse is down and what its position is. Finally, the boid object holds details about the window which is passed to the update function to check for wall collisions.

The BoidsColony object is a wrapper around the Boid behavior which acts as a container for the boids. It iterates over the boids by sets the steering behavior and then calls update and draw, mimicking the behavior of a single boid. The boid algorithm was constructed using the flee behavior for separation, seek for cohesion, and arrive for alignment. The flee vectors from each neighboring boid were divided by the distance from the neighboring boid to the current boid. This was added to a separation force vector which stores the cumulative weight of the flee vectors from the neighboring boids. The cohesion force is calculated similarly except the cohesion vector is from the character's position to the boid. This creates an opposite reaction to the flee behavior. Finally, the alignment behavior takes the sum velocity of the boids and subtracts from the current velocity. Then it calculates the average velocity by dividing by the total number of boids in the colony. Finally, if the boids aren't near any neighbors then the wandering behavior is called which causes the boid to go find neighbors.

Finally, the main loop handles the high level logic and takes any inputs from the keyboard to toggle the single Boid and BoidColony mode. The steering behaviors, speed, and smoothing param can all be adjusted using the keyword. The mouse is used for getting a target based on user input.

III. EXPERIMENT

A. Align Experiments

When performing tests with the align steering behavior, the lower smoothing parameters ended up looking more realistic than the high smoothing parameters because the

*This work was not supported by any organization

[1]Prem Gandhi is with Industrial and Systems Engineering Masters Program, North Carolina State University, Raleigh, NC 27606 phgandh2@ncsu.edu

boid was not as snappy which caused an unrealistic look . An ideal smoothing parameter was found around 0.0862706 which make the align behavior look more realistic, aligning in about 1 sec [Figure 9 & Figure 10]. The snappy align steering behavior was useful for handling the wall collisions as the boid could quickly be aligned in the opposite direction of the wall reducing the amount of time spent in further collisions.

B. Arrive Experiments

Experiments with the arrive steering behavior showed that decreasing the stop radius of the arrive behavior reduced the jerkiness of the boid. This parameter must be balanced with the time to arrive parameter which dictates how much the boid will slow down when getting close [Figure 1 & Figure 6]. The most successful arrivals were with the lower stop radius which reduced the amount of jerkiness and approached the target more gently [Figure 4]. However, the higher stop radius might be useful in scenarios where the arrival requires some distance between the target and the boid [Figure 3]. One example of a character interaction where distance between the target and the boid is necessary would be in NPC interactions or a creep behavior which would have the player rush in the beginning and slow later on.

C. Wander Steering Behaviors

Two different wander steering behaviors were implemented. The first wander steering behavior uses the kinematic orientation and adds random changes in orientation according to a random binomial distribution. This first wander steering behavior handles wall collisions by returning to the boid update loop which checks for a wall collision and then flees from the wall. This behavior is fickle because the kinematic wander doesn't change orientation that much because the random binomial distribution is biased towards outputting $0\,\mathrm{deg}$ [Figure 12]. The second wander steering behavior is an extension of the Align steering behavior and it projects a wander circle by a certain forward offset with a parameter for the radius of the circle. The algorithm handles wall collisions by predicting if a wall collision is imminent and then it delegates to the align behavior with a target at a random point within the window. The align steering is set with a smoothing parameter of 0 so that the boid immediately snaps to the the direction facing away from the wall. The selection of a random point within the window gives the appearance of a random change in behavior. The second wander algorithm performs much better as it doesn't get stuck as much and it has better wall collision handling by looking for another random point within the window. The second wander algorithm also performs more curvy movements around the screen as opposed to the kinematic wander which seems to only have small perturbations. The perturbation size could be increased but this would result in a wobbly movement towards the general direction of the boid. It could also perform a hockey stick shape movement. The second wander algorithm is able to perform more loop and windy movements that give the appearance of a map exploration behavior [Figure 13].

D. Boids Algorithm

The final objective of this project is to implement a boids algorithm that performs flocking behavior using blending/arbitration techniques. The boid algorithm was designed to have 2 major sets of variables: behavior weights and neighborhood radius for each behavior. The behavior weights were tuned to the following:

1) separation: 10.0
2) cohesion: .4
3) alignment: .1

Secondly, the neighborhood radius for each behavior was tuned as follows:

1) separation neighborhood radius: 20.0 pixels
2) cohesion neighborhood radius: 75 pixels
3) alignment neighborhood radius: 100 pixels

The behavior tuned to produce more organic looking behavior as collisions between boids have a fairly high but limited influence on the boids. The neighborhood radius for separation is small enough to ensure that boids don't hit each other and they will quickly flee if another boid enters its area [Figure 15]. The cohesion weight is substantially smaller than the separation, but it has a larger neighborhood radius so it's effect will operate on the boid in a short-medium time interval [Figure 16]. The alignment weight is the smaller and it has an effect in the long term as small flocks start to merge. This produces a longer term effect on the boids and it has more influence as the boids start to get in formation [Figure 17]. The number of boids spawned also has an effect on the convergence time of the algorithm. The boids seemed to be too jittery whenever there are 200 or more boids spawned. This may be due to the fact that the density is so high so the parameters must be tuned differently [Figure 19]. The parameters above are tuned for 50 boids which creates a flocking behavior that can effectively travel around the whole one in natural looking movements.

IV. CONCLUSIONS

In conclusion, this report demonstrates that basic steering behaviors can be implemented using object representations like Static, KinematicSteeringOutput, and KinematicMovements to hold important geometric information about game objects e.g. boids. These movement data objects can be passed to steering behaviors to produce natural looking movements based an object's current state and a target object's state. The importance of tuning for parameters is demonstrated in the experiments section where various parameter tunings can be implemented for different scenarios based on the context and use case such as a potential Creeping behavior using Arrive. Finally, the various simple steering behaviors can be combined using techniques like blending/abitration to make complex flocking movements (i.e. Boids) that have natural looking behavior. These algorithms maintain an O(1) due to the use of simple calculations,

providing high efficiency as the number of game objects scale up.

ACKNOWLEDGMENT

Dr. Tiffany Barnes thank you for instructing us in how to implement the various algorithms detailed in this report.

V. APPENDIX

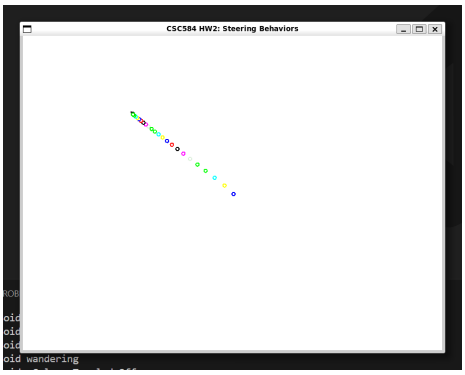

Fig. 1: Straight line

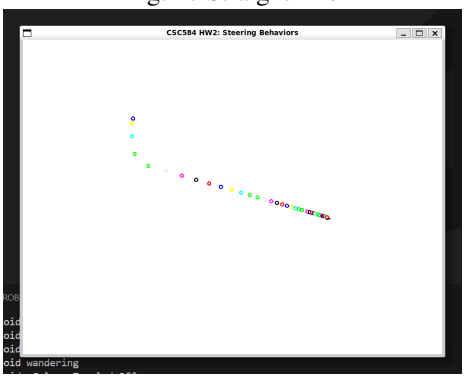

Fig. 2: Curved path

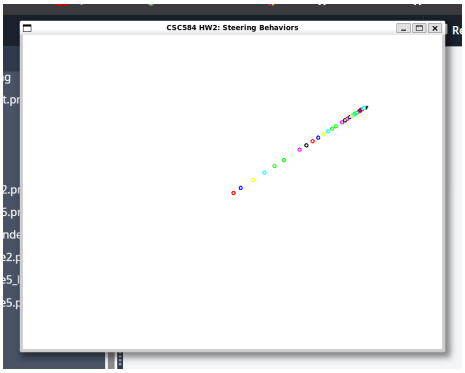

Fig. 3: Stop radius = 1.0f

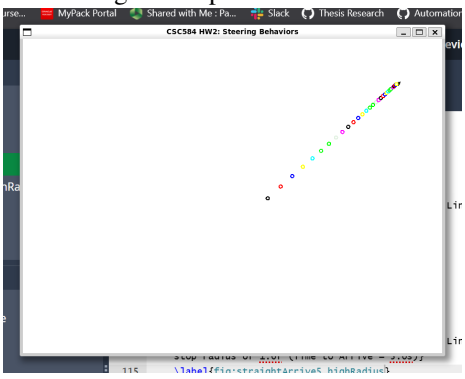

Fig. 4: Stop radius = 0.1f

Fig. 5: Arrive Steering Behavior (Time to Arrive = 5.0s)

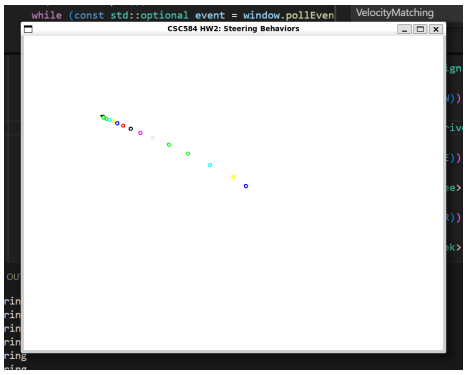

Fig. 6: Straight line

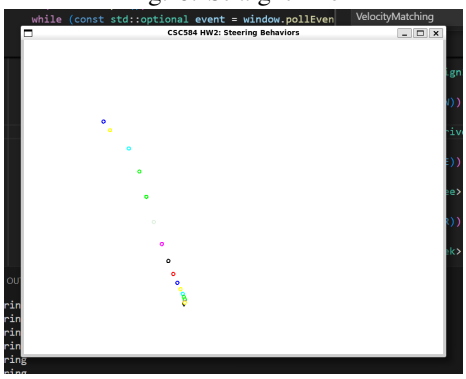

Fig. 7: Curved path

Fig. 8: Arrive Steering Behavior (Time to Arrive = 2.0s)

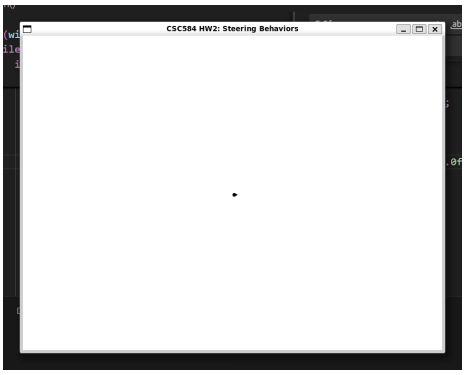

Fig. 9: Before align

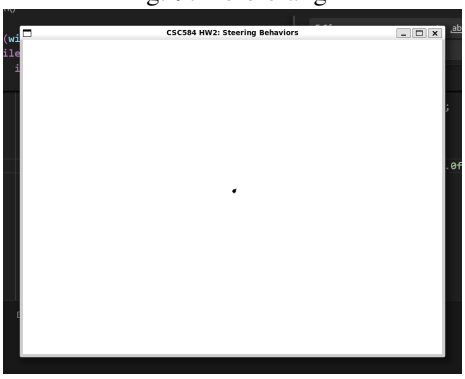

Fig. 10: After align

Fig. 11: Align Steering Behavior

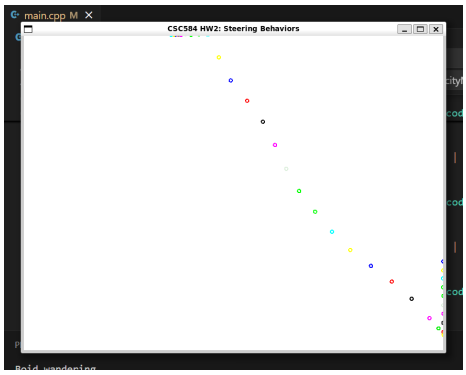

Fig. 12: Kinematic wander hitting wall

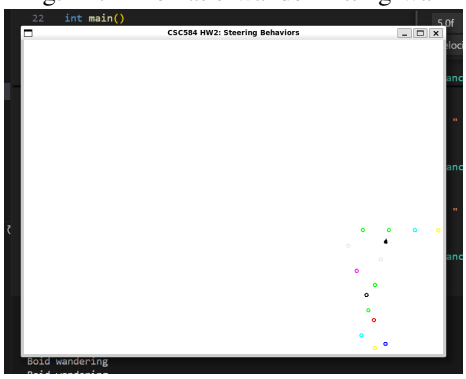

Fig. 13: Wander avoiding walls

Fig. 14: Comparison of Wander Behaviors

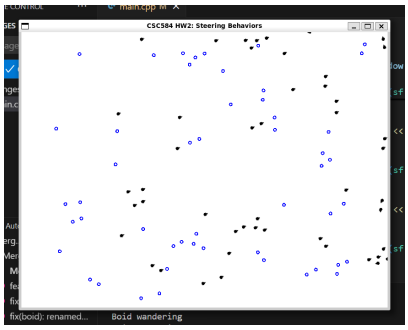

Fig. 15: Initial state

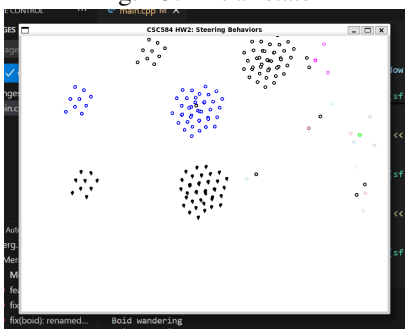

Fig. 16: Midway

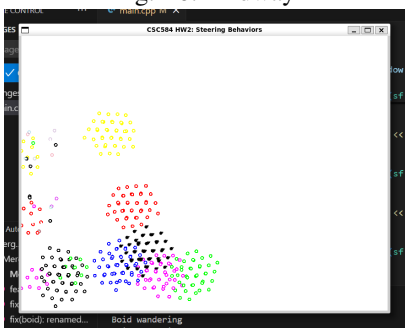

Fig. 17: Flocking together

Fig. 18: Boids Algorithm Progression

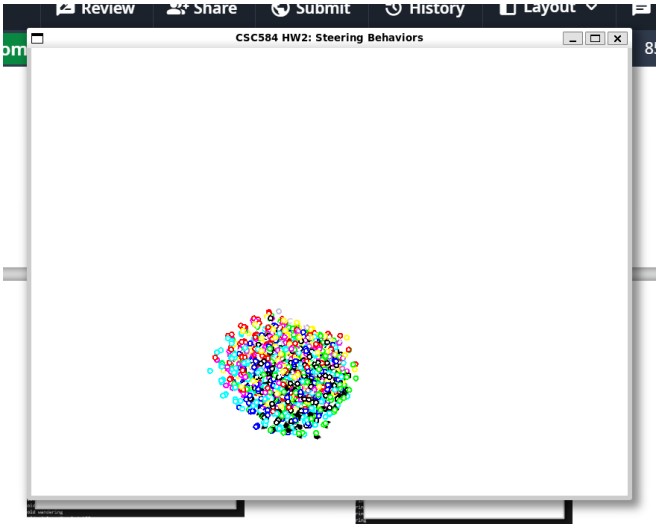

Fig. 19: 200 boids spawned in with clustered flocking behavior